Funktioner - Spørgsmål

Spørgsmål: Funktioner 1 - Kodens læsbarhed

Scenarie: Martin har set noget kode, der virker, men som er svær at læse. "*Programmet functions1.c udskriver antal elementer, men det er så svært at forstå! Hvad ville gøre det mere læsbart? Koden må ikke ændre funktionalitet.*" siger han.

Hvad du allerede ved:

- Kommentarer i C: `//` og `/* */`
- Navngivningskonventioner for variabler
- Funktioners opbygning
- Formatering med `printf`

Opgaven:

Programmets output er:

```
N(211)=15
N(93)=9
```

1. Undersøg koden i `functions1.c`
2. Find ud af, hvad der ville forbedre læsbarheden
3. Besvar: Hvad gør koden mere læsbar uden at ændre dens funktionalitet?
4. **PBL:** Martin spørger: "*Kan du forklare, hvorfor meningsfulde variabelnavne er vigtige i programmering?*"

Hint:

- Meningsfulde variabelnavne (f.eks. `values_above_element` i stedet for `x`)
- Kommentarer, der forklarer kode, som ikke er indlysende
- Ensartet indrykning
- Fornuftig brug af mellemrum
- Se koden i `functions1.c`

Spørgsmål: Funktioner 2 - Funktion til BMI-beregning

Scenarie: Martin vil gerne beregne BMI, men han vil ikke have formlen stående i `main()`. "*Jeg har brug for en funktion, der beregner BMI. Kan du lave en?*" spørger han. "*Den skal tage vægt og højde og returnere BMI.*"

Hvad du allerede ved:

- Funktionsdefinition og -deklaration
- Funktionsparametre
- Returværdier
- BMI-formlen: `bmi = weight / (height * height)`

Opgaven:

1. Definer variablerne `weight = 31.1` og `height = 1.31` i `main()`
2. Definer funktionen `float calculateBMI(...)` over `int main()`
3. I funktionen `calculateBMI`:
    - Implementer BMI-algoritmen
    - Returner BMI-værdien
4. Kald `calculateBMI` i `main` med `height` og `weight` som inputparametre
5. Gem den returnerede værdi i en `float bmi`-variabel
6. Udskriv resultatet
7. **PBL:** Martin spørger: "*Kan du ændre funktionen, så den også udskriver en kategori (Underweight, Normal, Overweight, Obese) ud fra BMI?*"

Hint:

- Hvilke datatyper skal inputparametrene have? `float` til begge
- Hvilken returtype skal funktionen have? `float`
- Funktionssignatur: `float calculateBMI(float height, float weight)`
- Kald: `float bmi = calculateBMI(height, weight);`
- BMI-kategorier: Underweight < 18.5, Normal 18.5-24.9, Overweight 25-29.9, Obese >= 30

Spørgsmål: Funktioner 3 - Udskriv værdi med pointer

Scenarie: Martin vil gerne udskrive en variabels hukommelsesadresse og værdi. "*Jeg har brug for en funktion, der tager en pointer til en int og udskriver adressen og værdien. Kan du lave den?*" spørger han.

Hvad du allerede ved:

- Pointers
- Funktionsparametre (pointer-typer)
- Adresseoperatoren &
- Dereference-operatoren *
- Format specifieren `%p` til adresser

Opgaven:

1. Definer variablen `int value = 101;` i `main`

2. Definer funktionen `void printValue(int *pval)` over `main`
3. I funktionen `printValue`:
    - Implementer udskrivningen, så den viser: `address: 0xF213d23, value: 101`
4. Kald `printValue(&value)` fra `main`
5. Hvilken information bliver sendt til `printValue`?
6. Kør programmet
7. **PBL:** Martin spørger: "*Kan du ændre det, så det også udskriver pointerens størrelse i bytes?*"

Hint:

- Sådan udskriver du en hukommelsesadresse: brug format specifieren `%p`
- Sådan dereferencer du en pointer: brug `*` (f.eks. `*pval`)
- Send adressen med &: `printValue(&value)`
- Informationen, der sendes, er hukommelsesadressen (pointerens værdi)

Spørgsmål: Patient 1 - Udskrivningsfunktion (call-by-value)

Scenarie: Martin vil gerne organisere sin kode, der udskriver patientoplysninger. "*Jeg har patientoplysninger, der skal udskrives, men det hele står i main(). Kan du lave en funktion, der klarer det?*" spørger han.

Hvad du allerede ved:

- Funktionsdefinition og -deklaration
- Funktionsparametre (call-by-value)
- `printf`-funktionen
- Flere parametre

Opgaven:

1. Lav en funktionsdeklaration og -definition `printPatientInfo(...)`
2. Start stille og roligt med én enkelt parameter: `id` (linje 11-12)
3. Erstat `printf`-linjerne i `main` med kald til `printPatientInfo`
4. Bemærk: `int main` må ikke indeholde nogen `printf`-kald

Startkode:

```
#include <stdio.h>
int main() {
    int id = 13;
    int age = 44;
    float weight = 80;
    float height = 1.80;
    printf("Patient Information:\n");
```

```
    /* Udskriv patient information */
    printf("###\n");
    printf(" PatientID:\t%d\n", id);
    printf(" age:\t\t%d years\n", age);
    printf(" weight:\t%.1f kg\n", weight);
    printf(" height:\t%.2f m\n", height);
    return 0;
}
```

5. **PBL:** Martin spørger: "*Kan du tilføje alle patientparametrene til funktionen og fjerne alle printf fra main?*"

Hint:

- Start med kun `id`-parameteren
- Tilføj derefter `age`, `weight` og `height` én ad gangen
- Funktionen skal indeholde alle printf-statements
- Kald: `printPatientInfo(id, age, weight, height);`

Spørgsmål: Patient 2 - Flere patienter med BMI

Scenarie: Martin vil gerne udskrive flere patienter med BMI-beregning. "*Jeg har et array af patienter. Kan du bruge udskrivningsfunktionen og tilføje BMI-beregning?*" spørger han.

Hvad du allerede ved:

- Arrays
- Funktioner
- Loops
- BMI-beregning

Opgaven:

1. Brug `printPatientInfo`-koden fra `patient1.c`
2. Lav en funktion `calculateBMI`, der beregner `bmi = weight / (height * height)`
3. Ændr `printPatientInfo`, så den udskriver BMI

Startkode:

```
#include <stdio.h>
int main() {
    int ids[] = {1, 3, 52, 23};
    int ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
    int number_of_patients = sizeof(ids) / sizeof(ids[0]);
    printf("Patient Information for %d patients\n", number_of_patients);
    for (int i = 0; i < number_of_patients; i++) {
```

```
        /* Udskriv patient information */
        printf("###\n");
        printf(" PatientID:\t%d\n", ids[i]);
        printf(" age:\t\t%d years\n", ages[i]);
        printf(" weight:\t%.1f kg\n", weights[i]);
        printf(" height:\t%.2f m\n", heights[i]);
    }
    return 0;
}
```

4. **PBL:** Martin spørger: "*Kan du også udskrive BMI-kategorien for hver patient?*"

Hint:

- Husk funktionsdeklarationen og -definitionen af `printPatientInfo`
- Beregn BMI: `bmi = calculateBMI(heights[i], weights[i])`
- Udskriv BMI med: `printf(" BMI:\t\t%.1f\n", bmi);`

Spørgsmål: Records - Udskrivning med call-by-reference

Scenarie: Martin vil gerne udskrive data fra en struct ved hjælp af en funktion. "*Jeg har en Record-struct med id, dato, noter og en måling. Kan du lave en funktion, der udskriver den?*" spørger han.

Hvad du allerede ved:

- Struct-definition
- Funktionsparametre (call-by-reference)
- `ctime()`-funktionen til formatering af datoer
- Pointers til structs

Opgaven:

1. Definer en struct-skabelon `Record`, der indeholder:
    - `id`: integer
    - `dato`: `time_t`
    - `notes`: string på 100 char
    - `measurement`: float
2. Lav funktionsdeklarationen og -definitionen `print_record(Record *)` (call-by-reference)
3. Funktionen skal:
    - Udskrive en læsbar dato ud fra `dato` (se `ctime(t)`)
    - Eksempel på output for 1594974180: `date: Fri Jul 17 10:23:00 2020`
    - Udskrive alle de øvrige oplysninger
4. Definer følgende i `main`:

```
struct Record rec = {131, 1594974180, "Measurement recorded without issues",
93.3};
```

5. Kald `print_record` med `rec` som call-by-reference
6. Kør programmet
7. **PBL:** Martin spørger: "*Kan du udvide det, så det udskriver flere records fra et array?*"

Hint:

- Send structens adresse: `&rec`
- Brug `ctime(&record.dato)` til at få en læsbar dato
- Inkluder `time.h` for at bruge `ctime()`
- Funktionssignatur: `void print_record(struct Record *r)`

Spørgsmål: Patient 3 - Statistikfunktioner

Scenarie: Martin vil gerne beregne statistik for flere patienter. "*Jeg har brug for funktioner til gennemsnit og standardafvigelse. Kan du lave dem, så de kan genbruges?*" spørger han.

Hvad du allerede ved:

- Arrays
- Funktioner
- Loops
- Statistiske formler
- Matematikbiblioteket

Opgaven:

1. Skriv funktionen `printStatistics(char* measurement, float gennemsnit, float afvigelse)` - den skal udskrive målingens navn, gennemsnit og standardafvigelse
2. Skriv funktionen `float mean(float x[], int n)` - beregner gennemsnittet $\mu = \frac{1}{N}\sum_{t=0}^{n-1} x_t$
3. Skriv funktionen `float std(float x[], int n)` - beregner standardafvigelsen $\sigma = \sqrt{\frac{1}{N-1}\sum_{t=0}^{n-1}(x_t - \mu)^2}$
4. Brug funktionen `printStatistics`

Startkode:

```
#include <stdio.h>
int main() {
    int ages[] = {13, 44, 51, 100};
    float weights[] = {80, 99, 100, 50};
    float heights[] = {1.80, 1.62, 1.30, 1.74};
```

```
    int number_of_patients = sizeof(ages) / sizeof(ages[0]);
    printf("Patient Descriptives for %i Patients\n", number_of_patients);
    return 0;
}
```

5. **PBL:** Martin spørger: "*Kan du ændre det, så det beregner og udskriver statistik for alle tre målinger (alder, vægt og højde) i ét loop?*"

Hint:

- Du har lavet disse ligninger før
- Gennemsnit: $\mu = \frac{1}{N} \sum x_t$
- Standardafvigelse: $\sigma = \sqrt{\frac{1}{N-1} \sum (x_t - \mu)^2}$
- Husk at inkludere `<math.h>` for at bruge `sqrt()`